



Teton Talk

Technical interview brief

A push-to-talk walkie-talkie for care-home staff that keeps working on patchy wifi and never quietly loses what someone said.

CONTENTS

- 1 What it is, and the 60-second pitch
- 2 System topology & repo layout
- 3 The app: layers and wiring
- 4 State management (Redux Toolkit + SQLite)
- 5 The talk lifecycle, end to end
- 6 Network state machine & resilience
- 7 Wire protocol
- 8 The server
- 9 Audio pipeline
- 10 The simulator & tests
- 11 Trade-offs, weak spots and likely questions

What it is

Teton Talk is a walkie-talkie for care-home staff. You hold one big button to talk live to everyone on the channel ("Floor 1"). Everything said is also **recorded and kept as a message**, so someone who was away, offline or busy can catch up. The brief graded three things: it keeps working through patchy wifi and short dropouts, it never quietly loses what someone said, and it is easy to use.

The 60-second pitch

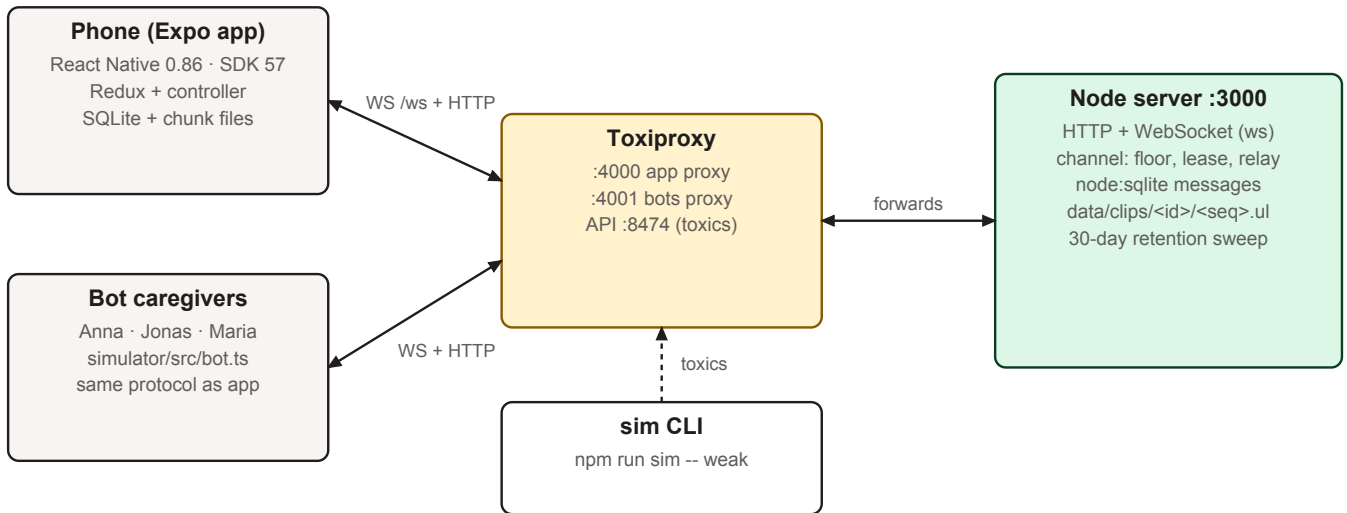
"The phone's recording is the source of truth. Every 250 ms of audio is written to disk before it goes anywhere. Live streaming over a WebSocket is an optimisation layered on top of a guaranteed, resumable HTTP upload. The server commits a message only when every chunk is present, and the clip id is the idempotency key, so a message is delivered whole and exactly once or it stays on the sender's phone as 'Not sent yet' and keeps retrying. Listeners never get a partial clip. On top of that sits a pure network state machine that absorbs short blips so the UI doesn't flicker, and a simulator with Toxiproxy and bot caregivers that proves all of this with automated tests."

Key numbers to remember

Constant	Value	Why
Codec	8 kHz G.711 μ -law, 1 byte/sample (64 kbps)	Pure JS, no native codec, robust; also runs on server & bots
Chunk size	250 ms = 2,000 bytes	Unit of streaming, disk writes, upload and acks
Jitter buffer	300 ms	Small network hiccups don't chop live audio
Floor lease	3 s	Speaker who goes silent / drops frees the floor
Absorb window	3 s	Drops shorter than this never change the UI state
Recovering band	5 s	"Back online — N missed" then settles
Weak thresholds	RTT > 800 ms, 2 missed pongs, backlog > 1.5 s	Back to online only below 500 ms (hysteresis)
Heartbeat	ping every 2 s; 3 missed = reconnect	Catches zombie sockets NetInfo can't see
Reconnect backoff	1, 2, 4, 8, 15 s $\pm$ 50% jitter	Avoid a ward of phones stampeding the AP
Min / max clip	300 ms / 60 s	Accidental taps discarded; hard cap with a buzz
Late flag	> 30 s between recorded and committed	"Sent N min ago" tag
Retention	30 days, swept hourly	Server deletes old clips

Topology and repo layout

Four packages share one repo. The **shared/** folder is imported by the app, the server, the bots and the tests, so they can never disagree about the protocol, the codec or the network rules.

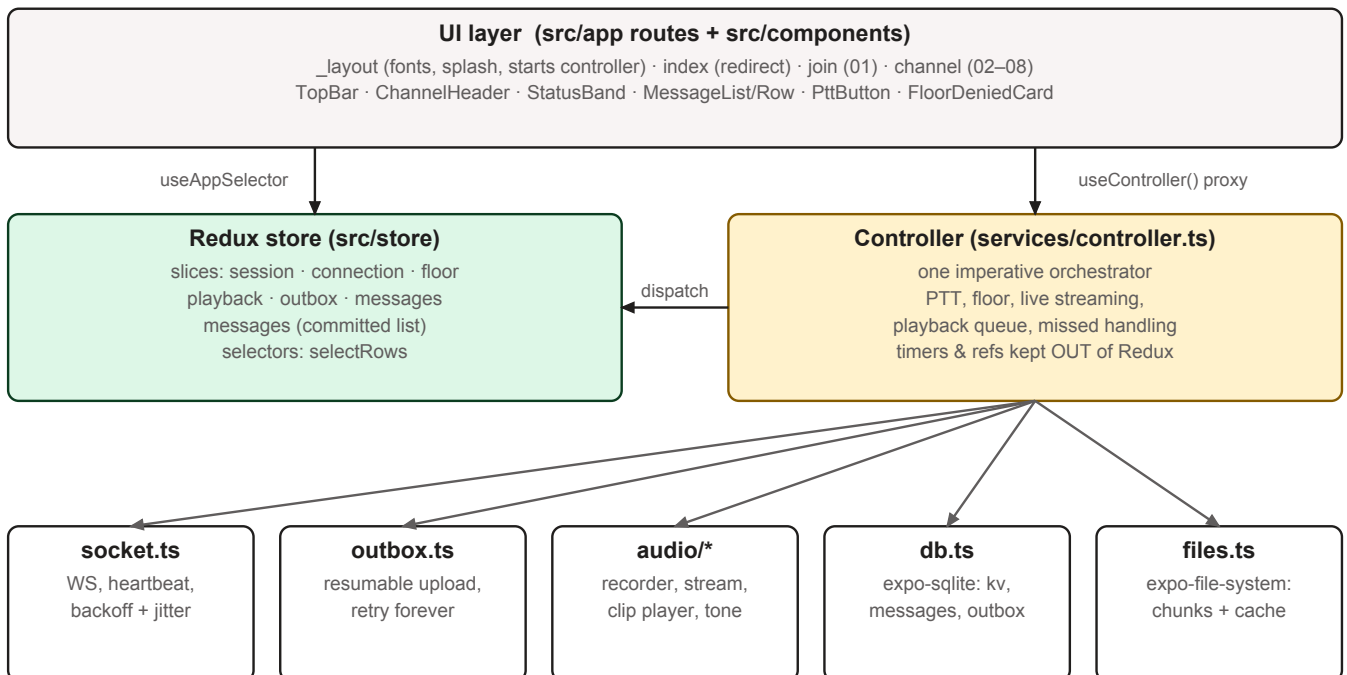


In development the app **always** talks to the server through Toxiproxy (port 4000), so you can degrade the link at will. The bots use a separate proxy (4001), which lets you break the phone's link and the bots' links independently. The server URL is picked in `src/config.ts`: `localhost:4000` on the iOS simulator, `10.0.2.2:4000` on the Android emulator, or `EXPO_PUBLIC_SERVER_URL`.

Repo layout

Path	What lives there
<code>src/app/</code>	Expo Router routes: <code>_layout.tsx</code> (fonts, splash hold, starts controller), <code>index.tsx</code> (redirect), <code>join.tsx</code> (screen 01), <code>channel.tsx</code> (screens 02–08, all states on one screen)
<code>src/components/</code>	TopBar, ChannelHeader, StatusBand (weak/offline/back-online bands), MessageList + MessageRow, PttButton, FloorDeniedCard, Splash, Logo
<code>src/services/</code>	controller (orchestrator), socket (WS + heartbeat + backoff), outbox (resumable upload), db (expo-sqlite), files (expo-file-system), registry
<code>src/audio/</code>	recorder, streamPlayer, clipPlayer, tone, context (wrappers over react-native-audio-api)
<code>src/store/</code>	Redux Toolkit store, 6 slices, listener middleware for persistence, memoised selectors
<code>src/theme/tokens.ts</code>	Design tokens copied exactly from the design handoff
<code>shared/</code>	<code>protocol.ts</code> (types, constants, binary frames), <code>mulaw.ts</code> (codec + resampler), <code>netMachine.ts</code> (pure reducer)
<code>server/src/</code>	<code>index.ts</code> (HTTP + WS boot, retention), <code>channel.ts</code> (presence, floor, lease, relay, commit), <code>http.ts</code> (upload API), <code>db.ts</code> , <code>clips.ts</code>
<code>simulator/</code>	Toxiproxy client + network scenarios (CLI), bot caregivers + REPL, Vitest resilience tests, voice samples
<code>plugins/withSceneLifecycle.js</code>	Config plugin: makes the generated iOS project adopt the <code>UIScene</code> life cycle (iOS 27 crashes at launch otherwise)

Layers and how they are wired



The app has a clear split: **Redux holds what the UI renders**, and a single **controller** holds the imperative machinery (sockets, timers, the active recording, audio nodes). Components never touch the socket or audio directly; they read the store and call controller methods.

Boot sequence

- `_layout.tsx` wraps everything in the `Redux Provider`, loads the Archivo fonts and keeps the splash up for at least 1.2 s and at most 2 s. Screens mount only after fonts register (an iOS quirk keeps the fallback font otherwise).
- Slices hydrate synchronously from SQLite at store creation (`session` loads name, `clientId`, `lastSeq`, `serverOffset`; `playback` loads the MISSED list).
- Once a name exists, `createController(store).start()` runs: restore the outbox from SQLite, check/ask mic permission, open the socket.
- `index.tsx` redirects to `/join` (first launch) or `/channel`.

Notable wiring patterns (good interview talking points)

- **Controller as a factory with closures** (`createController`, `createSocket`, `createOutbox`): no classes, dependencies passed in (`dispatch`, `getState`, `callbacks`). Easy to reason about and to fake.
- **registry.ts** holds the single controller instance. It is created in an effect once the user has a name, so screens and hooks look it up there at call time.
- `useController()` returns a stable `Proxy` that resolves `registry.controller` at call time, because the controller is created inside an effect and may not exist on a screen's first render. The proxy identity never changes, so it is safe in `useCallback` deps.
- **channel.tsx is one screen for designs 02–08**: it derives a `PttState` discriminated union (`idle` / `pending` / `live` / `local` / `receiving` / `micOff`) and the status band from store state. No navigation between states.
- **Store subscription for mode downgrade**: the controller subscribes to the store; if the network becomes weak or offline while you're talking, `floor.my.mode` flips to `local` (stop streaming, keep recording, send after).
- **NetInfo is only a hint**: OS "disconnected" drops the socket immediately, "connected" retries immediately, but the source of truth is the server's `welcome` and heartbeats.

Redux Toolkit and SQLite

Tools: **Redux Toolkit 2** (`createSlice`, `configureStore`, `createListenerMiddleware`, `createSelector`), **react-redux 9** typed hooks `useAppSelector` / `useAppDispatch`, and **expo-sqlite** for durability.

Slice	Holds	Persisted?
<code>session</code>	name, <code>clientId</code> (random UUID on first launch), <code>lastSeq</code> (catch-up cursor), <code>serverOffset</code> (server clock – phone clock)	Yes, via listener middleware → kv table
<code>connection</code>	Everything from <code>NetState</code> (the pure machine) + online count, <code>onlineAtDrop</code> , <code>nextRetryAt</code> , <code>missedOnReturn</code> . Its <code>netEvent</code> reducer just calls the shared <code>netReducer</code> .	No
<code>floor</code>	<code>speaker</code> (someone else live), <code>holding</code> (finger down), my <code>{clipId, startedAt, mode: pending live local}</code> , <code>denied</code> (lost race), <code>level</code> (meter), <code>notice</code> , <code>micDenied</code>	No
<code>playback</code>	current <code>{messageId, positionMs, durationMs, playing}</code> , <code>queue</code> (auto-play order), <code>missed</code> (ids tagged MISSED)	missed only
<code>outbox</code>	Mirror of the SQLite outbox: my clips the server hasn't committed + progress (0..1) for "Sending N%"	Yes, written by outbox service
<code>messages</code>	The committed message list, ordered by server seq; <code>heardBy</code> counts	Yes, via listener middleware → messages table

Why there is no RTK Query

- The message list is a plain `messages` slice. Its `initialState` loads from SQLite (pruned to a 12 h shift), so the list renders instantly, even offline.
- After that it is updated **by socket events**: the controller dispatches `upsertMessages(list)` and `setHeardBy({messageId, heardBy})`. The listener middleware writes those changes back to SQLite, so the reducers stay pure.
- An earlier version used RTK Query with `fakeBaseQuery`, infinite cache lifetime and mutations that only called the controller. That used none of RTK Query's real features (fetching, cache expiry, refetching, loading states), so it was replaced by this slice. Retry and delete now call the controller directly, like play already did.
- When to bring it back: if messages came from a real paginated REST endpoint. Here, catch-up already arrives over the socket (`welcome.missed`).

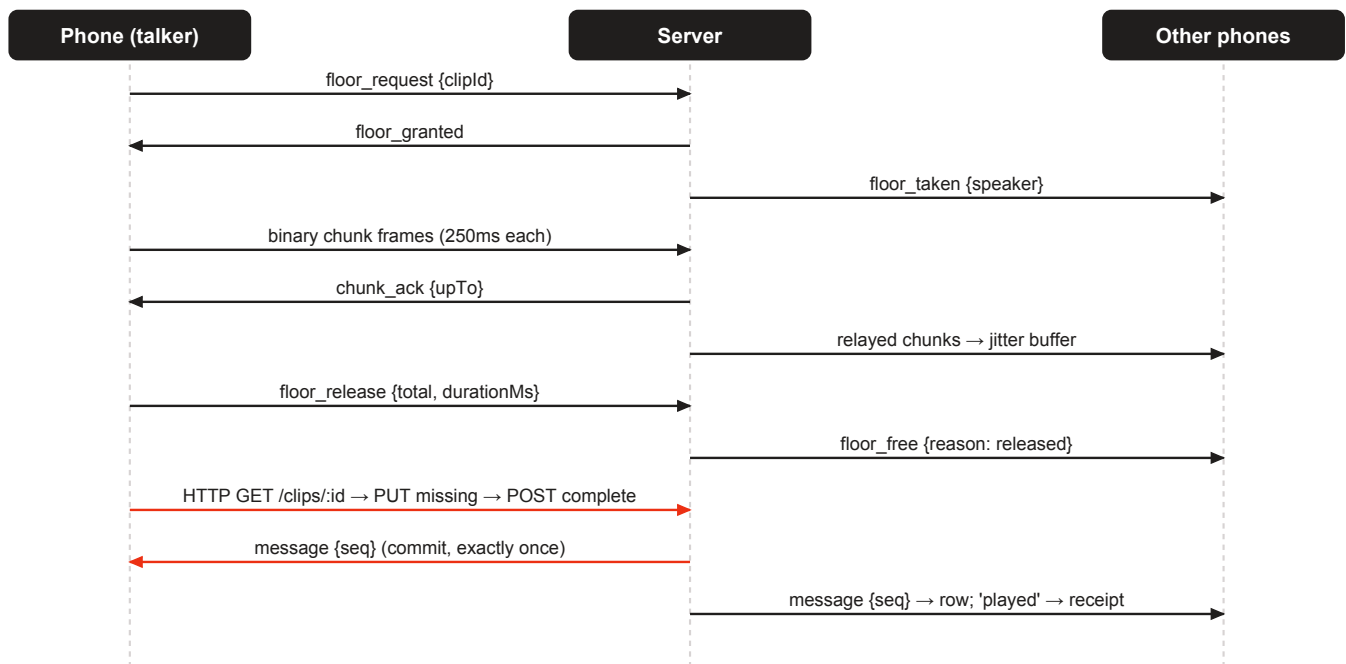
Derived state

- `selectRows` (`createSelector`) merges committed messages, the outbox, the missed list and network state into one `Row[]` with exactly **one receipt per row**: time / Heard by N / Not sent yet / Sending N% / Not sent.
- Unsent clips are appended at the bottom in recording order, so the newest thing is always next to the button.
- Timestamps are shown as `recordedAt - serverOffset`: all times are in server clock, converted to the phone clock only for display.

What is deliberately *not* in Redux

Timers, the active recording (`clipId`, `sentUpTo`, `recorded`, `grantTimer`, `maxTimer`), the last live ack, resume position, audio nodes and the socket instance. They change many times a second or aren't serialisable. This is why the store disables `serializableCheck` and `immutableCheck` (also for speed on mobile).

The talk lifecycle, end to end



Step by step

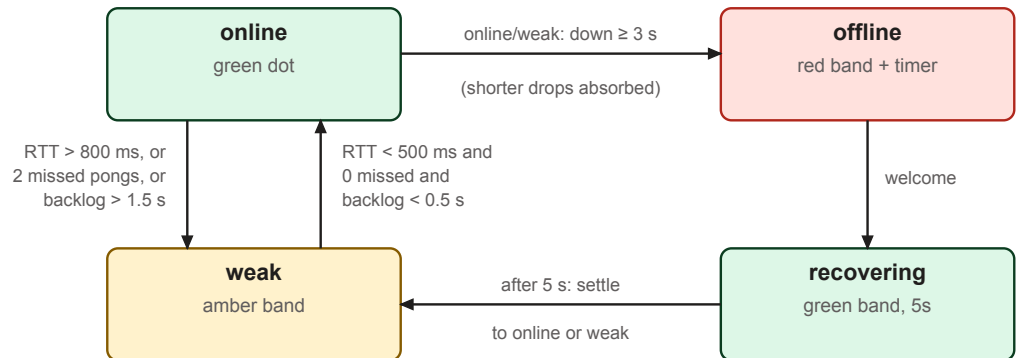
- **pressIn** → `setHolding(true)`. If someone else is live, do nothing yet (you'll go live when they stop).
- **beginTalking**: new `clipId (UUID)`, `recordedAt` in server clock, outbox row created with status `recording`, mic starts, 60 s max timer. Mode is `pending` if the link is good, otherwise `local`.
- Every chunk: **written to disk first** (`clips/<id>/<seq>.ul`), then sent as a binary frame only if mode is `live`.
- `floor_request` goes out. If no answer in 1 s → mode `local` (record-and-send). On `floor_granted`: haptic + tone, mode `live`, and chunks recorded during the wait are flushed from disk.
- `floor_denied` (lost a split-second race): recording is **discarded**, double buzz, card 08. If you keep holding, you go live automatically on `floor_free`.
- **pressOut** → `endTalking`: under 300 ms is discarded with a hint. Otherwise `outbox.finish(total, duration)`; if live, send any tail chunks and `floor_release` (server may commit immediately), then kick the outbox after 1.5 s.
- **Outbox**: `GET /clips/:id` → `PUT` only missing chunks → `POST` complete. 409 = some missing, retry resumes. Network errors retry forever with backoff (2→30 s); 4xx and unreadable audio become "Not sent · Retry".
- Server broadcasts `message`; everyone upserts it. The talker's outbox drops the clip. Listeners who heard every chunk live send `played` immediately; others get it tagged **MISSED** and auto-played oldest first.

Half duplex and pre-emption

Talking mutes the stream player. Live incoming talk interrupts catch-up playback, remembers the position, and resumes when the floor frees. Playback that is interrupted halfway keeps the **MISSED** tag; only playing to the end clears it.

Network state machine and failure handling

`shared/netMachine.ts` is a **pure reducer**: $(\text{NetState}, \text{NetEvent}) \rightarrow \text{NetState}$. Inputs are facts (`link_up`, `link_down`, `pong{rtt}`, `ping_missed`, `backlog{ms}`, `tick`); output is one of four states. Because it's pure and shared, it is unit-tested in Node with synthetic clocks, and the app just feeds it events (plus a 250 ms tick).



- **Link up = server's** `welcome`, not TCP open. A socket through a dead proxy or captive portal can open and carry nothing.
- **Absorb**: state doesn't change until a drop has lasted 3 s; `offlineSince` is backdated to the actual drop so the offline timer is honest.
- **Hysteresis**: weak above 800 ms RTT, back to online only below 500 ms, so a borderline link doesn't flap.
- **Cold start without signal** counts as "down since launch" → offline after 3 s instead of a hanging splash.

Failure matrix (from RESILIENCE.md)

Failure	Behaviour	Proof
Wifi blip < 3 s	Absorbed; socket reconnects, outbox fills gaps	netMachine test; sim patchy
Offline for longer	Red band, timer, saved count, retry countdown; talking still records locally	sim offline / dropout
Upload dies halfway	Next attempt asks what the server has and sends only the rest	test: resumes
Connections reset repeatedly	Backoff retries, still exactly one message	test: reset_peer
Speaker's link dies mid-talk	3 s lease frees floor; listeners told "message will arrive in full"; full clip arrives later	test: speaker cut
Zombie socket	2 missed pongs = weak, 3 = forced reconnect	sim zombie
Weak link while talking	Stops streaming, keeps recording, "Sending N%"	sim weak
App killed mid-recording	On relaunch, contiguous chunks on disk are sent, tagged "Cut short"	outbox.restore()
Two people press at once	Server: first arrival wins; loser records nothing	test: floor race; bots race
Away for a while	<code>hello{lastSeq}</code> → server returns exactly the missed messages	test: lastSeq
Late delivery	Server-clock timestamps; > 30 s late gets a tag	test: late flag

The wire protocol (shared/protocol.ts)

One WebSocket at `/ws` for control and live audio, plus a small HTTP API for the guaranteed path. JSON text frames carry control messages; binary frames carry audio. Both sides use TypeScript discriminated unions (`ClientMessage`, `ServerMessage`) so every switch is exhaustively typed.

Client → Server	Purpose
hello {clientId, name, lastSeq}	Identify; lastSeq drives catch-up
floor_request {clipId, recordedAt}	Ask to go live
floor_release {clipId, total, durationMs, recordedAt}	Stop; total 0 = discard (tap / lost race)
played {messageId}	Read receipt → "Heard by N"
ping {sentAt}	Heartbeat; RTT + clock offset
Server → Client	Purpose
welcome {online, floor, serverTime, missed[]}	Link is truly up; catch-up list
presence {online}	Online count
floor_granted / floor_denied / floor_taken / floor_free {reason}	Floor control; reason = released lease_expired disconnected
chunk_ack {clipId, upTo}	Highest contiguous chunk stored (backlog signal)
message {ChannelMessage}	A committed message
receipt {messageId, heardBy}	Receipt count changed
pong {sentAt, serverTime}	Heartbeat reply

Binary audio frame

```
[0]      frame type (1 = audio chunk)
[1..36]  clipId, ASCII UUID
[37..40] seq, uint32 big-endian
[41..]   μ-law payload (2,000 bytes for 250 ms)
```

HTTP API (resumable upload + history)

```
GET  /health          -> { ok, online }
GET  /messages?since=<seq> -> { messages }
GET  /clips/:id       -> { received: number[], committed }
PUT  /clips/:id/chunks/:seq body: μ-law bytes (idempotent)
POST /clips/:id/complete -> { message } | 409 { missing: number[] }
GET  /clips/:id/audio  -> whole clip, audio/basic
Headers: x-client-id, x-client-name
```

Clock handling

Each pong gives `serverTime + RTT/2`; the phone stores `serverOffset` (persisted). Clips recorded offline are stamped in server time, so "late" means really late, and ordering across phones is consistent. The server also clamps `recordedAt` to not be in the future.

The server (server/src)

Node 22.13+ with TypeScript run by `tsx`, the `ws` library for WebSockets, the plain `node:http` module for HTTP, and the **built-in** `node:sqlite` (no native build). Only one runtime dependency: `ws.createServer({port, dataDir})` is exported so the tests can boot a real server in a temp directory.

channel.ts: the heart

- **Presence:** `Map<WebSocket, Client>`; online count is distinct `clientIds`. On `hello`, a stale socket for the same `clientId` is terminated (a reconnect can beat the old socket's close).
- **Floor:** one in-memory `floor` object. First `floor_request` to arrive wins (Node's single thread makes this atomic). Others get `floor_denied` with who has it.
- **Lease:** a 250 ms interval frees the floor if the holder sent no audio for 3 s (`lease_expired`). Socket close frees it too (`disconnected`).
- **Relay:** every binary chunk is stored and acked; it's relayed to others **only** if it's from the floor holder for the current clip, so listeners never hear two people.
- **Contiguous acks:** per-clip set of seen seqs, `upTo` advances while the next seq exists; seeded from disk after a restart.
- **commit()** is the only way a message is created: return existing if already committed; else 409 with missing seqs; else assemble the file and `INSERT OR IGNORE`. Broadcast only when actually created.

Storage

- **SQLite (WAL):** `messages(seq AUTOINCREMENT, id UNIQUE, sender, duration, recorded_at, committed_at)` and `receipts(msg_id, client_id) PK.seq` is the global order and the catch-up cursor; `heard_by` is a COUNT subquery. A sender can't mark their own message heard (enforced in SQL).
- **Clips on disk:** `data/clips/<id>/<seq>.ul`, written to a `.tmp` then renamed (atomic, so a half-written chunk never counts). Assembled into `<id>.ul` at commit.
- **Retention:** hourly sweep deletes messages, receipts and files older than 30 days.
- **Defensive limits:** clip ids must match a UUID regex (prevents path traversal), max 64 KB per WS chunk, 256 KB per HTTP body, seq 0–10,000, names truncated to 40 chars.

Audio pipeline (src/audio + shared/mulaw.ts)

Built on **react-native-audio-api** (a Web Audio API implementation for RN, by Software Mansion). It's needed because the app requires *raw PCM from the mic while recording* and *a queue of buffers it can append to while playing*. File-based recorders/players such as expo-audio don't expose that. It contains native code, so the app needs a **development build**, not Expo Go.

File	Role
context.ts	One shared <code>AudioContext</code> ; iOS session <code>playAndRecord</code> + <code>defaultToSpeaker</code> + Bluetooth HFP. <code>mulawBuffer()</code> decodes μ -law and upsamples 8 kHz to the context rate (otherwise it plays ~6× fast and sounds like noise).
recorder.ts	<code>AudioRecorder.onAudioReady</code> at 16 kHz, 100 ms buffers → resample to 8 kHz (device may not honour the rate) → slice into exact 2,000-sample chunks → μ -law → <code>onChunk(seq, bytes)</code> . Also emits an RMS level for the meter (~11 Hz).
streamPlayer.ts	Live incoming talk: <code>AudioBufferQueueSourceNode</code> , 300 ms jitter buffer before starting, dedupes seqs, mutes while you talk, counts chunks heard per clip ("heard it all live" vs MISSED).
clipPlayer.ts	Recorded clip playback: <code>AudioBufferSourceNode</code> with offset (resume), 100 ms progress ticker, token guard so a replaced clip's <code>onEnded</code> doesn't fire.
tone.ts	Synthesised 880 Hz "you're live" blip (oscillator + gain envelope); no asset needed.
shared/mulaw.ts	G.711 μ -law encode/decode with a 256-entry decode table, a resampler (box-filter averaging when downsampling, linear interpolation when up), RMS helpers. Pure JS so app, server and bots share it.

Why μ -law at 8 kHz?

Voice-band quality is fine for a walkie-talkie, the codec is ~50 lines of pure JS, every chunk is a fixed 2,000 bytes, and any 250 ms chunk decodes independently (no codec state across chunks), which makes resumable upload and dedup trivial. The cost is bandwidth (64 kbps vs 16–24 kbps for Opus), which is the first thing to change at scale.

The simulator, bots and tests

The simulator is what proves the resilience claims instead of asserting them. It has three parts.

Network chaos: Toxiproxy (Shopify)

A TCP proxy with an HTTP API that injects "toxics". `simulator/src/toxiproxy.ts` is a thin client; `npm run sim -- <scenario>` applies a preset (each starts from a clean link; Ctrl-C restores). `--target app|bots|all` picks which proxy to degrade.

Scenario	Toxics	What it should show
weak	600±400 ms latency both ways, 6 KB/s upstream (below the 8 KB/s stream)	Amber band, record-and-send, "Sending N%"
offline	Proxy disabled until good	Red band, timer, retry countdown, "Not sent yet"
patchy	2 s cut every 10 s	Band must never flicker (absorb)
dropout	One 8 s cut	Offline → Back online
flapping	Random 1–6 s cuts	Stability under churn
zombie	timeout toxic both ways: socket open, nothing flows	Heartbeat detects it
reset	reset_peer 1.5 s after each connect	Uploads resume, not restart

Bot caregivers

Bot (EventEmitter) speaks the **same protocol as the app**, including its own outbox and resumable upload, so tests exercise real recovery paths. `npm run bots` starts Anna, Jonas and Maria with a REPL: `anna talk 6`, `jonas leave`, `auto on`, `status`, and `race`, which adds 1.5 s downstream latency to the phone so you genuinely lose a floor race (screen 08). Voices come from prerecorded μ -law samples.

Automated tests (Vitest)

The harness boots the **real server** in a temp dir, starts `toxiproxy-server` if needed, and gives each bot its own proxy so one person's link can be broken at a time.

Suite	Cases
Never quietly lose	Recorded with no signal → delivered exactly once · interrupted upload resumes · reset_peer still delivers once · speaker cut mid-stream frees the floor within the lease and the full clip arrives later · reconnect with lastSeq returns exactly the missed messages
Floor control	Two people press at once: one wins, the loser records nothing
Server idempotency	Duplicate chunks and completes commit once · > 30 s is flagged late · complete with missing chunks → 409 with the list
netMachine (pure)	Absorbs 2 s dropout · offline after 3 s with backdated offlineSince · recovering 5 s · cold start → offline · RTT hysteresis · missed pongs / backlog → weak
Codec	μ -law round-trip within quantisation error · 16→8 kHz halves length · chunk frames encode/decode

Gap to own: there are no React component or controller unit tests in the app; coverage is protocol- and server-level, plus manual QA with the simulator.

Trade-offs, weak spots and likely questions

Things to fix or be ready to defend

- **No authentication.** The server trusts `x-client-id / hello.clientId`; anyone who can reach it can impersonate a user or upload clips. Answer: device enrolment per care home + signed tokens; TLS (Android currently allows cleartext for dev).
- **Single process, in-memory floor.** Can't scale horizontally. Answer: floor state in Redis with a TTL lease, Postgres + object storage for clips, sticky or pub/sub fan-out.
- **The controller is a ~500-line orchestrator.** It works and has one place for all side effects, but could be split (talk, playback, catch-up) or modelled as an explicit state machine (e.g. XState) and unit-tested with a fake socket.
- **Outbox is head-of-line:** on a network error it stops the whole flush and waits (deliberate, to avoid hammering a bad link). Chunks are PUT one by one, sequentially; could be pipelined.
- **µ-law bandwidth** (64 kbps): Opus would keep weak links live longer. WebRTC/LiveKit for the live path, keeping this upload as the guarantee.
- **Playback progress and level meters run on JS timers** (100–250 ms), so a busy JS thread makes them stutter. Reanimated shared values would move that work off the JS thread.
- `outbox.restore()` hardcodes `total * 250 ms` and the controller hardcodes `8000` in `chunkCountFor`: small duplications of protocol constants.
- **Out of scope on purpose:** background/locked-screen audio (iOS `UIBackgroundModes`, Android foreground service), volume-key PTT, multiple channels.

Likely questions, short answers

Question	Answer
How do you guarantee nothing is lost?	Disk first, then wire. Every clip goes through the outbox, live or not. Server commits only with all chunks; clipId is the idempotency key. Outcome is "delivered once" or "visibly not sent yet, retrying".
Why WebSocket + HTTP, not just one?	WS is low-latency for live audio and control; HTTP gives simple, stateless, resumable, idempotent upload that is easy to retry and to reason about.
How do you avoid duplicates?	<code>INSERT OR IGNORE</code> on a <code>UNIQUE</code> clip id; chunk writes are idempotent; client dedupes by message id and stream seq.
How is the floor race resolved?	Server-authoritative: first <code>floor_request</code> processed wins (single-threaded Node). Loser's recording is discarded and they auto-go-live when the floor frees if still holding.
What if the speaker disappears?	3 s lease. Listeners hear what made it live, get a notice, and later receive the full clip via the speaker's outbox.
Why Redux and not Zustand/Context?	Many independent events (socket, timers, audio) update shared state that several components read; RTK gives typed slices, Immer, selectors, listener middleware for persistence, and great devtools. The controller lives outside React so it isn't tied to render cycles.
Why is netMachine a pure reducer?	Deterministic and testable with fake clocks; shared by app and tests; the UI band is a pure function of facts.
How do you detect a weak network?	Measured, not guessed: RTT from pings, missed pongs, unacknowledged live audio (<code>sentUpTo</code> – <code>ackedUpTo</code>), with hysteresis.
Why a development build?	<code>react-native-audio-api</code> is a native module; Expo Go only bundles its own modules. Built with <code>expo run:ios android</code> or EAS.
How are ios/android handled?	Continuous Native Generation: config in <code>app.json</code> + config plugins (audio-api permissions, build properties, the custom <code>UIScene</code> plugin). Native folders are generated, not hand-edited.
What would you do next?	Opus/WebRTC, background audio + push wake-up, auth, multi-instance server, metrics (time-to-commit, fallback rate, retries), and a real-device pilot on a ward.

UX decisions worth mentioning

- Mic denied → outlined button "MICROPHONE OFF · Allow microphone", tap opens Settings.
- Talking on a bad link is **not green** (green means people hear you now): outlined "RECORDING · Sends when complete".
- Holds under 300 ms → "Hold the button to talk"; 60 s cap with amber timer in the last 10 s.
- Haptics: heavy buzz on mic open, double buzz on losing the race; 44 pt hit targets; accessibility labels and live-region bands.
- Gender-neutral copy ("the moment they stop"); forced light mode; long-press the brand to rename.